

Evaluating A Model Report

Introduction

This project uses machine learning to predict messages as either spam or ham (not spam). The goal is to make a model that can identify spam messages and not incorrectly predict legit messages as spam.

Dataset

Dataset name & link: [SMS Spam Collection](#)

Total Messages: 5,572

Ham: 4,825

Spam: 747

Model

Scikit learn

Model & link: [Logistic Regression](#)

Preparing data

Training/Testing Split: 80% Training

20% Testing

Repeatable: random state 42 to compile the same

Model's Brain: TF-IDF was imported to convert the text messages into numerical numbers. The model was only trained on the converted training data not on the converted test data to avoid data leakage.

Training Environment

Python version: 3.14.3

Scikit-learn: 1.9.1

Time to train: 0.02206897735595703

Confusion Matrix:

	Pred. Ham	Pred. Spam
Ham	[[966(tn)	0(fp)]
Spam	[28(fn	121(tp)]]

Metrics:

Accuracy: 0.9748878923766816

Precision: 1.0

Recall: 0.8120805369127517

F1 Score: 0.8962962962962964

Specificity: 1.0

The arithmetic was hard coded in the project and sklearn.metrics was commented out but both shown.

```
accuracy1 = (tp + tn) / (tp + tn + fp + fn)
precision1 = tp / (tp + fp)
recall1 = tp / (tp + fn)
f1 = (precision1 * recall1) / (precision1 + recall1) * 2
specificity = tn / (tn + fp)
```

Baseline: 0.8663677130044843
Improvement: 0.10852017937219727

Which metric matters the most?

The metric that matters most for the problem could be two different ones and for different reasons. It would change if you're looking at how it performed in the training or if this was a hypothetical software. Recall would be the most important metric for how it performed in training. This is because we want to catch as many spam messages as possible. The model had a recall of 81.21% meaning that it correctly identified a solid amount of spam messages. The false negatives that got passed the model would mean it predicted not spam when it was a spam message.

On the flip side of that the model had no false positives in training. That would mean a non-spam message got sent to spam when it wasn't. If this was a software and a user never saw the message because it is in spam would make precision the most important metric because we want to avoid that as much as possible. In the hypothetical I would rather have a user delete a spam message that got through the software then the same user miss the content of an important message if a false positive happened.

Is accuracy misleading on a dataset?

Accuracy was misleading in my training. This is because my dataset was imbalanced. My dataset of SPAM messages and NOT spam messages had way more NOT spam messages. There were 4,825 ham messages and 747 spam messages. This means the model could have predicted ham every single time and would still get close to 86% accuracy on this specific test set. The model's accuracy of 97.49% accuracy doesn't seem that impressive when you know the dataset. That is why the other metrics matter significantly.

Where does the model fail?

The model had 28 false negatives (FN's), meaning it said ham when the message was spam. One concrete example it got wrong was, "Reminder: You have not downloaded the content you have already paid for. Goto <http://doit.mymoby.tv/> to collect your content". The correct label is spam but the model predicted ham. I speculate it got this one wrong because honestly as a human I look at that message and see it going either way. Or even if you bought something from a business that has auto text messages those could be important and not spam. I could also see that this spam message doesn't have many typical spam message words. So the model wouldn't have the high numbers it is trained to look for. The link couldn't be a good give away for spam but the model isn't trained on those things yet just common words or spam messages.

Is the model better than baseline?

The baseline model would always predict ham because that is the majority class of our data set. On our test dataset the 966 ham messages / 1115 total messages would give us 86.64% accuracy. The trained model achieved an accuracy of 97.49%. Therefore, the trained model improved its accuracy by about 10.85% over the baseline model. The trained model would also beat a random model. With the dataset you have two options (ham/spam), giving you a simple 50/50 guess would be approximately in the long run 50% accuracy. That is significantly lower than the trained model.